

PROYECTO FINAL FIA

Buscando al coronel Kurtz

Resumen

En esta memoria se presenta el proyecto final de la asignatura de **Fundamentos de Inteligencia Artificial**. El trabajo se estructura en **dos partes** (Palacio lógico y Palacio bayesiano) e incluye además un problema de **planificación como MDP** (cruce del río). Se abordan los siguientes temas:

- **Descripción de la aproximación al problema:** se emplean técnicas de **búsqueda** e **inferencia** estudiadas en el curso. En la Parte 1 se implementa un **agente lógico** que construye una base de conocimiento a partir de percepciones y decide acciones mediante búsqueda (BFS). En la Parte 2 se desarrolla un **agente bayesiano** que mantiene distribuciones de probabilidad (trampas, militar y salida), actualiza creencias con regla de Bayes y planifica con **A*** penalizando el riesgo. Finalmente, el cruce del río se modela como **MDP** y se resuelve con **Value Iteration**, extrayendo la política óptima.
- **Diseño de la interfaz:** la interacción se realiza mediante una **interfaz de consola estilo “arcade”**, con paneles y tableros impresos, permitiendo visualizar tanto el “mapa mental” del agente lógico como el **heatmap** de probabilidades del agente bayesiano y la **política** del MDP del río.

La resolución del problema se ha implementado en **Python**. Este proyecto consta (según los módulos entregados) de los siguientes elementos de desarrollo software:

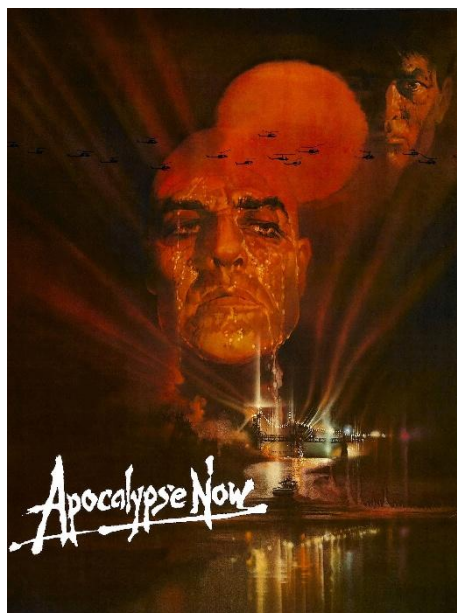
- **Módulos:** kurtz.py (programa principal o *main*), palacio_logico.py, palacio_bayesiano.py, rio_mdp.py, utilidades_visuales.py.
- **2053 líneas de código** (suma de los 5 módulos .py anteriores).
- **75 funciones y métodos** (incluye métodos de clases y algunas funciones internas).
- **10 clases.**
- **Librerías:** dependencia externa principal **numpy**; y librerías estándar como random, heapq, collections, os, sys, time, re.

INTRODUCCIÓN

Este proyecto, “**Buscando al Coronel Kurtz**”, plantea una misión inspirada en *Apocalypse Now* en la que el **Capitán Willard** debe localizar y rescatar al **Coronel Kurtz** dentro de un palacio peligroso y, posteriormente, **escapar** superando un entorno aún más incierto. A lo largo del trabajo se abordan dos escenarios complementarios que permiten aplicar, de forma integrada, distintos paradigmas de **Inteligencia Artificial** para la toma de decisiones con información parcial: primero, el razonamiento lógico y la búsqueda en un entorno discreto; y después, el razonamiento probabilístico y la planificación óptima bajo incertidumbre.

En la **primera parte**, el problema se modela como exploración de una cuadrícula (6×6) donde el agente debe evitar peligros (trampas y un soldado enemigo), interpretar **perceptos locales** (brisa, ronquidos, resplandor, paredes, etc.) y actuar con seguridad hasta encontrar a Kurtz y alcanzar la salida. Esto se resuelve mediante un **agente basado en conocimiento** (lógica proposicional) y, opcionalmente, mediante algoritmos de **búsqueda** (BFS) para automatizar la navegación a partir de las celdas inferidas como seguras.

En la **segunda parte**, el proyecto evoluciona hacia escenarios donde la certeza deja de ser posible: se introduce un **agente bayesiano** con distintos tipos de trampas y estímulos, que exige mantener y actualizar **creencias a posteriori** para estimar riesgos y guiar decisiones; y, finalmente, se modela el cruce de un río como un **Proceso de Decisión de Markov (MDP)**, donde la dinámica estocástica del entorno se resuelve mediante **Value Iteration** para obtener políticas óptimas que equilibren seguridad y eficiencia. En conjunto, la memoria documenta el modelado, los algoritmos implementados, las decisiones de diseño y el análisis de resultados obtenidos en ambos entornos.



1. PARTE 1 · Palacio lógico (deducción + búsqueda)

1.1. Modelado del entorno y creación de mapa

El entorno del problema se implementa como un **palacio 6×6** con colocación aleatoria de elementos (precipicios, militar, Kurtz y salida) respetando que el inicio sea seguro. Esta responsabilidad recae en la clase *EscenarioPalacio*, que actúa como “mundo real”: conoce el estado completo del mapa y es el único componente que puede generar perceptos fieles a ese estado.

La inicialización se realiza en `__init__`, donde se preparan tanto la posición del agente como banderas de estado y se delega la generación aleatoria a `_construir_escenario`. Este método construye la lista de celdas disponibles excluyendo (0,0), las baraja y asigna posiciones a cada elemento (3 precipicios, 1 militar, 1 coronel y 1 salida), cumpliendo el requisito del enunciado de generar un mapa distinto en cada ejecución.

Como fundamento común para perceptos y movimientos, `_calcular_celdas_adyacentes` define la vecindad de tipo Manhattan (N/S/E/O). Esta noción es esencial porque todas las sensaciones del enunciado dependen de “estar en una celda contigua”.

```
class EscenarioPalacio:
    def _construir_escenario(self):
        todas_celdas = [(fila, col) for fila in range(self.dimension)
                        for col in range(self.dimension)]
        # Excluir la posición inicial del capitán
        todas_celdas.remove((0, 0))

        random.shuffle(todas_celdas)

        # Asignar elementos
        self.ubicaciones_precipicios = [todas_celdas.pop() for _ in range(3)]
        self.ubicacion_militar = todas_celdas.pop()
        self.ubicacion_coronel = todas_celdas.pop()
        self.ubicacion_escape = todas_celdas.pop()

    def _calcular_celdas_adyacentes(self, fila, columna):
        """Retorna las celdas adyacentes válidas (Manhattan distancia 1)"""
        candidatas = [
            (fila - 1, columna), # Norte
            (fila + 1, columna), # Sur
            (fila, columna - 1), # Oeste
            (fila, columna + 1), # Este
        ]
        validas = []
        for f, c in candidatas:
            if 0 <= f < self.dimension and 0 <= c < self.dimension:
                validas.append((f, c))
        return validas
```

1.2. Perceptos: generación y traducción a salida comprensible

El enunciado establece un vector de perceptos que informa sobre peligros cercanos y paredes, además del grito al eliminar al militar. La generación del vector se concentra en *EscenarioPalacio.obtener_sensaciones*, que calcula:

- **Brisa** si hay precipicio en alguna adyacente.
- **Ronquido** si el militar está en una adyacente y sigue vivo.
- **Resplandor** si la salida está en la celda actual o adyacente.
- **Paredes** según bordes del tablero.
- **Grito** tras la eliminación del militar.

Así, el agente no “ve” el mapa real: solo recibe sensaciones consistentes con su posición, como exige el planteamiento.

Para mejorar la trazabilidad, *interpretar_sensaciones_logicas* transforma el vector booleano en salida textual coloreada (capa de presentación, no de lógica).

```
def obtener_sensaciones(self):
    """
    Retorna los perceptos del estado actual
    [Brisa, Ronquido, Resplandor, ParedNorte, ParedEste, ParedSur, ParedOeste, Grito]
    """
    fila, columna = self.posicion_willard
    celdas_vecinas = self._calcular_celdas_adyacentes(fila, columna)

    # Detectar brisa (precipicios adyacentes)
    hay_brisa = any(ubicacion in self.ubicaciones_precipicios
                    for ubicacion in celdas_vecinas)

    # Detectar ronquido (militar adyacente y vivo)
    hay_ronquido = False
    if not self.militar_eliminado:
        hay_ronquido = any(ubicacion == self.ubicacion_militar
                           for ubicacion in celdas_vecinas)

    # Detectar resplandor (salida adyacente o en la misma celda)
    zona_resplandor = celdas_vecinas + [(fila, columna)]
    hay_resplandor = any(ubicacion == self.ubicacion_escape
                          for ubicacion in zona_resplandor)

    # Detectar paredes
    en_borde_norte = (fila == 0)
    en_borde_este = (columna == self.dimension - 1)
    en_borde_sur = (fila == self.dimension - 1)
    en_borde_oeste = (columna == 0)

    # Grito (se activa una vez después de matar al militar)
    hay_grito = self.escuchar_grito
    self.escuchar_grito = False

    return [hay_brisa, hay_ronquido, hay_resplandor,
            en_borde_norte, en_borde_este, en_borde_sur, en_borde_oeste,
            hay_grito]
```

	0	1	2	3	4	5	LEYENDA
0	CW	E?	??	??	??	??	CW = Willard CW+K = Willard+Kurtz
1	E?	??	??	??	??	??	SAL! = Salida
2	??	??	??	??	??	??	P! = Precipicio
3	??	??	??	??	??	??	S! = Soldado
4	??	??	??	??	??	??	P/S/E? = Sospecha
5	??	??	??	??	??	??	OK = Seguro ?? = Desconocido

Sensaciones: Resplandor, (Muros: N,W)

Ejemplo de sensación de resplandor y posible salida.

1.3. Agente lógico: base de conocimiento y deducción

El núcleo de la Parte 1 es el agente lógico implementado como *CerebroLogico*.

Este agente mantiene una **base de conocimiento incremental**: no decide únicamente por el último percepción, sino por el historial acumulado.

La base de conocimiento se estructura con conjuntos para:

- celdas **exploradas** y **confirmadas seguras**,
- memoria de señales (brisa, ronquido, resplandor),
- conjuntos de **posibles** y **confirmados** (precipicios/militar),
- y salida (*salidas_posibles*, *salida_confirmada*).

El método clave es *procesar_informacion(posicion_actual, sensaciones)*, que:

1. marca la celda actual como explorada y segura,
2. restringe o descarta hipótesis de **salida** según resplandor,
3. añade o elimina candidatos de **precipicios** según brisa,
4. añade o elimina candidatos de **militar** según ronquido,
5. confirma deducciones con *_deducir_peligros_exactos* (si queda un único candidato) y amplía seguras con *_marcar_celdas_seguras* (si no hay señales).

Finalmente, *visualizar_conocimiento* imprime un “mapa mental” del agente distinguiendo certeza, sospecha y seguridad, aportando explicabilidad sin revelar el mapa real.

```
def procesar_informacion(self, posicion_actual, sensaciones):...

def _deducir_peligros_exactos(self, ubicaciones_sensacion, confirmados, posibles):
    """Si solo hay un candidato para una sensación, debe ser el culpable"""
    for ubicacion_sensacion in ubicaciones_sensacion:
        f, c = ubicacion_sensacion
        vecinos = self._obtener_vecinos(f, c)
        candidatos = [v for v in vecinos if v not in self.celdas_confirmadas_seguras]

        if len(candidatos) == 1:
            culpable = candidatos[0]
            confirmados.add(culpable)
            if culpable in posibles:
                posibles.remove(culpable)
```

	0	1	2	3	4	5	LEYENDA
0	L	SAL!	??	??	??	??	CW = Willard CW+K = Willard+Kurtz
1	CW	OK	??	??	??	??	SAL! = Salida
2	OK	??	??	??	??	??	PI = Precipicio
3	??	??	??	??	??	??	SI = Soldado
4	??	??	??	??	??	??	P/S/E? = Sospecha
5	??	??	??	??	??	??	OK = Seguro ?? = Desconocido

Salida deducida en: (0, 1)

Salida deducida tras percepción previo.

1.4. Piloto automático: BFS sobre celdas seguras

El enunciado propone planificar mediante búsqueda. Aquí se implementa una planificación incremental, donde el agente decide el siguiente paso en función de su conocimiento.

decidir_movimiento_automatico(posicion_actual, algoritmo="BFS"):

- prioriza la granada si el militar está confirmado en un vecino,
- define objetivo (Kurtz, salida, o exploración de seguras no exploradas),
- y realiza búsqueda restringida a *celdas_confirmadas_seguras* mediante BFS (*_buscar_amplitud*).

Este diseño cumple el requisito de explorar y llegar a metas evitando deliberadamente celdas no seguras según el conocimiento interno del agente.

```
def decidir_movimiento_automatico(self, posicion_actual, algoritmo="BFS"):
    """
    Decide la siguiente acción usando búsqueda
    Retorna: comando ('w', 'a', 's', 'd', 'e', 'g*', o None)
    """
    # 1. Si hay militar confirmado adyacente, lanzar granada
    vecinos = self._obtener_vecinos(*posicion_actual)
    for vecino in vecinos:
        if vecino in self.militar_confirmado:
            diff_fila = vecino[0] - posicion_actual[0]
            diff_col = vecino[1] - posicion_actual[1]

            if diff_fila == -1: return 'gw'
            elif diff_fila == 1: return 'gs'
            elif diff_col == -1: return 'ga'
            elif diff_col == 1: return 'gd'

    # 2. Determinar objetivo
    objetivo = None
    if self.kurtz_localizado:
        if self.salida_confirmada:
            objetivo = self.salida_confirmada
    elif self.posicion_kurtz_conocida:
        objetivo = self.posicion_kurtz_conocida

    # 3. Ejecutar búsqueda
    siguiente_celda = None
    if algoritmo == "BFS":
        siguiente_celda = self._buscar_amplitud(posicion_actual, objetivo)
    elif algoritmo == "DFS":
        siguiente_celda = self._buscar_profundidad(posicion_actual, objetivo)

    # 4. Traducir a comando
    if siguiente_celda:
        if siguiente_celda == posicion_actual:
            if self.kurtz_localizado and posicion_actual == self.salida_confirmada:
                return 'e'
            return 'e' if (posicion_actual == self.salida_confirmada
                           and self.kurtz_localizado) else None

        diff_fila = siguiente_celda[0] - posicion_actual[0]
        diff_col = siguiente_celda[1] - posicion_actual[1]

        if diff_fila == -1: return 'w'
        if diff_fila == 1: return 's'
        if diff_col == -1: return 'a'
        if diff_col == 1: return 'd'

    if self.kurtz_localizado and self.salida_confirmada == posicion_actual:
        return 'e'

    return None
```

2. PARTE 2 · Incertidumbre: Bayes + MDP

2.1. Palacio bayesiano (inferencia probabilística + A*)

2.1.1. Entorno bayesiano: mapa y perceptos con estímulos múltiples

La Parte 2 introduce trampas (Fuego, Pinchos, Dardos) con estímulos detectables desde contiguas, permitiendo solapamientos. El entorno se implementa como *EscenarioPalacioBayesiano*.

_construir_escenario distribuye trampas (posible coincidencia entre ellas) y coloca Militar/Kurtz/Salida en celdas sin trampas, cumpliendo el requisito de que dichas entidades no se sitúan sobre trampas.

2.1.2. Motor de inferencia bayesiana.

El motor se encapsula en *MotorInferenciaBayesiana*, que mantiene distribuciones para fuego, pinchos, dardos, militar y salida. La inicialización sigue un **prior uniforme** y excluye (0,0) como celda candidata inicial.

actualizar_probabilidades(posicion_actual, sensaciones):

- aplica Bayes multiplicando por verosimilitudes consistentes/inconsistentes según vecindad (presencia/ausencia de estímulo),
- aplica evidencia fuerte (celda actual con probabilidad 0 de trampas; y si hay grito, militar se anula),
- y normaliza con *_normalizar_distribucion* para mantener distribuciones válidas.

Además, *calcular_riesgo_total(f,c)* combina probabilidades de muerte bajo independencia, produciendo un criterio único de riesgo para la planificación.

```

def actualizar_probabilidades(self, posicion_actual, sensaciones):
    """Actualiza todas las distribuciones usando el teorema de Bayes"""
    self.celdas_exploradas.add(posicion_actual)

    estimulo_fuego = sensaciones[0]
    estimulo_pinchos = sensaciones[1]
    estimulo_dardos = sensaciones[2]
    estimulo_militar = sensaciones[3]
    estimulo_salida = sensaciones[4]
    grito = sensaciones[9]

    # 1. Actualizar militar
    if grito:
        self.prob_militar = np.zeros((self.dimension, self.dimension))
    else:
        self._aplicar_regla_bayes(self.prob_militar, posicion_actual, estimulo_militar)

    # 2. Actualizar trampas
    self._aplicar_regla_bayes(self.prob_fuego, posicion_actual, estimulo_fuego)
    self._aplicar_regla_bayes(self.prob_pinchos, posicion_actual, estimulo_pinchos)
    self._aplicar_regla_bayes(self.prob_dardos, posicion_actual, estimulo_dardos)

    # 3. Actualizar salida
    self._aplicar_regla_bayes(self.prob_salida, posicion_actual, estimulo_salida)

    # 4. Evidencia implícita: estamos vivos en esta celda
    fila, columna = posicion_actual
    self.prob_fuego[fila, columna] = 0
    self.prob_pinchos[fila, columna] = 0
    self.prob_dardos[fila, columna] = 0
    if not grito:
        self.prob_militar[fila, columna] = 0

    # 5. Normalizar
    self._normalizar_distribucion(self.prob_fuego)
    self._normalizar_distribucion(self.prob_pinchos)
    self._normalizar_distribucion(self.prob_dardos)
    if not grito:
        self._normalizar_distribucion(self.prob_militar)
    self._normalizar_distribucion(self.prob_salida)

```

```

def _aplicar_regla_bayes(self, matriz_prob, posicion_observador, percibe_estimulo):
    """
    
$$P(\text{Elemento}_{ij} \mid \text{Evidencia}) = \frac{1}{Z} \cdot P(\text{Evidencia} \mid \text{Elemento}_{ij}) \cdot P(\text{Elemento}_{ij})$$

    Modelo determinista:
    - Si percibimos estímulo:  $P(\text{estímulo} \mid \text{elemento}_{ij}) = 1$  si  $ij$  en zona, 0 si no
    - Si NO percibimos:  $P(\neg \text{estímulo} \mid \text{elemento}_{ij}) = 0$  si  $ij$  en zona, 1 si no
    """
    fila, columna = posicion_observador
    zona_influencia = self._obtener_zona_influencia(fila, columna)

    for i in range(self.dimension):
        for j in range(self.dimension):
            esta_en_zona = (i, j) in zona_influencia

            if percibe_estimulo:
                # Solo las celdas en la zona pueden ser culpables
                verosimilitud = 1.0 if esta_en_zona else 0.0
            else:
                # Las celdas en la zona NO pueden ser culpables
                verosimilitud = 0.0 if esta_en_zona else 1.0

            matriz_prob[i, j] *= verosimilitud

def _normalizar_distribucion(self, matriz):
    """Normaliza para que sume 1"""
    suma_total = np.sum(matriz)
    if suma_total > 0:
        matriz /= suma_total

```

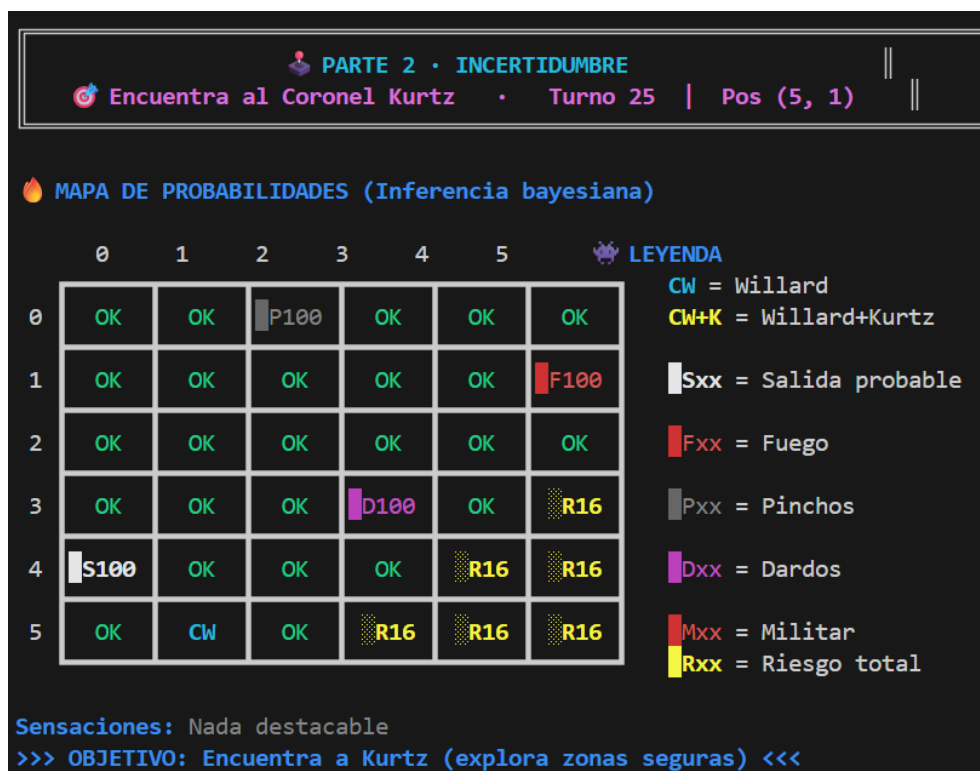
2.1.3. Decisión automática: umbral de riesgo + A* penalizado

La decisión automática se centra en:

- lanzar granada si $P(\text{militar})$ en un vecino supera un umbral,
- fijar objetivo (p. ej. máximo de $P(\text{salida})$ tras rescatar a Kurtz),
- planificar con `_buscar_a_estrella(...)` evitando celdas con riesgo alto y penalizando el riesgo en el coste.

2.1.4. Heatmap de creencias

`visualizar_heatmap(...)` imprime un mapa donde cada celda resume la creencia dominante (salida probable, amenaza más probable o riesgo), apoyándose en `_analizar_celda` para priorizar qué se muestra. Esto facilita interpretar cómo evolucionan las probabilidades y por qué el agente elige ciertas rutas.



Ejemplo ejecución.

2.2. Cruce del río como MDP (Value Iteration)

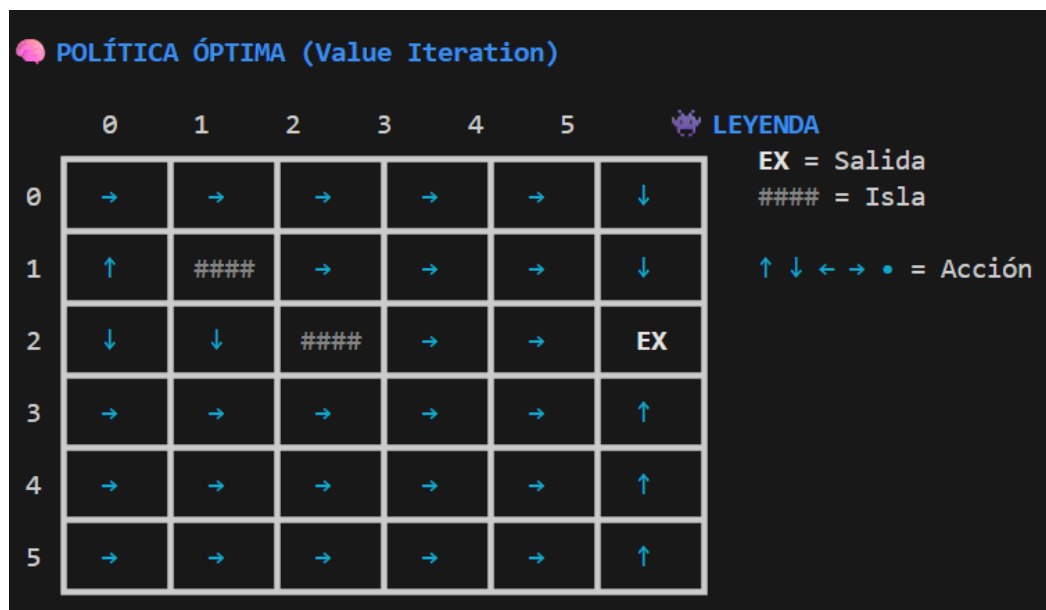
2.2.1. Representación del problema y visualización

El cruce del río se modela como MDP: estados (celdas), acciones (movimientos), transiciones probabilísticas afectadas por la corriente y recompensas. La visualización se realiza con `visualizar_rio(...)`, mostrando tablero, islas y salida.

2.2.2. Value Iteration y extracción de política

`SolucionadorMDP` implementa Value Iteration:

- `iterar_valores(gamma, epsilon)` aplica Bellman hasta que $\text{delta_maximo} < \text{epsilon}$.
- `_extraer_politica(gamma)` elige la acción que maximiza retorno esperado usando $V(s)$ convergido.
- `visualizar_politica()` imprime la política con flechas.

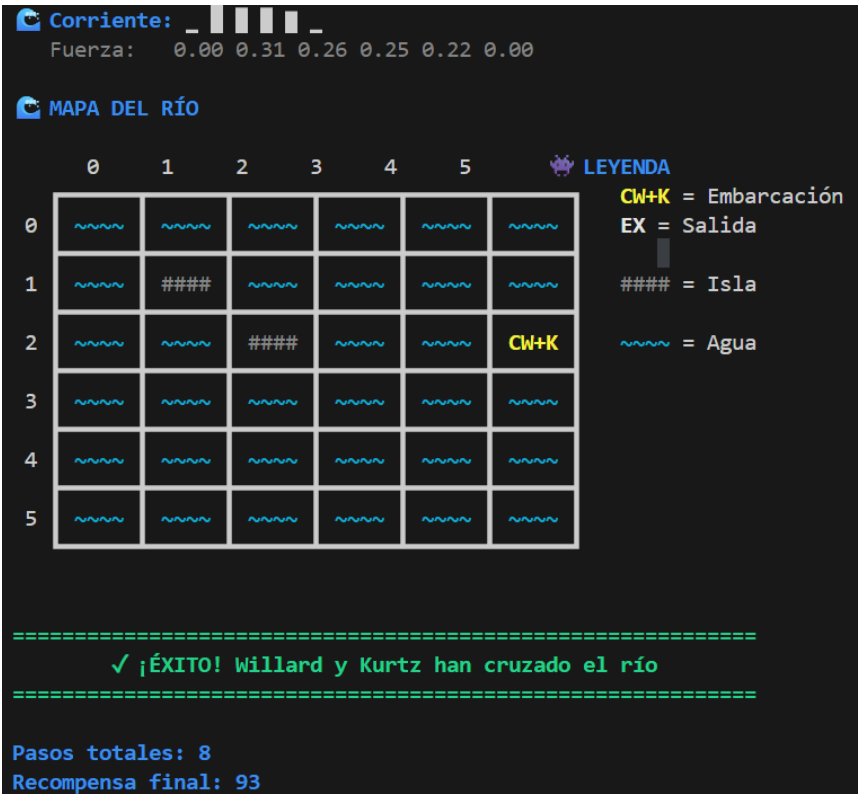


Ejemplo de política óptima en una ejecución.

2.2.3. Controlador de simulación

ControladorRioMDP.iniciar_simulacion() integra:

- 1. presentación del río,
- 2. cálculo de valores y política,
- 3. visualización de política,
- 4. simulación paso a paso aplicando la política.



Ejecución del MDP.

2.2.4. Análisis de resultados: influencia de probabilidades y “peligros” en la política óptima

La política óptima obtenida con **Value Iteration** no se limita a “ir hacia la salida”, sino que elige acciones según su **valor esperado**, teniendo en cuenta dos factores clave:

- **Probabilidades de transición (corriente):** en columnas con corriente fuerte, muchas acciones (left/right/up/stay) son menos fiables porque existe una alta probabilidad de **deriva hacia abajo**. Al calcular $Q(s, a)$ ponderando los posibles estados siguientes por su probabilidad, el algoritmo detecta que si la deriva suele llevar a estados peores, el **valor esperado baja** y la política tiende a **evitar** esas acciones o a usarlas solo cuando compensa.
- **“Peligros” u obstáculos (islas):** si el agente intenta moverse a una isla o fuera del mapa, **rebota** y se queda en el mismo estado, gastando un paso sin avanzar. Como hay **coste por paso** y recompensa al llegar a la salida, la política evita zonas con muchos rebotes porque aumentan los **pasos esperados** y reducen el retorno total.

La política final busca un **equilibrio**: maximiza la probabilidad de avanzar realmente y minimiza el número esperado de pasos, evitando áreas donde corriente u obstáculos vuelven las acciones demasiado arriesgadas o ineficientes.

3. Implementación conjunta

3.1. Utilidades comunes de visualización

La salida por consola se centraliza en *utilidades_visuales*, donde se definen estilos y funciones reutilizables:

- *EstilosConsola*
- *impresión arcade* (*panel_arcade*, *imprimir_tablero_arcade*, *imprimir_tablero_arcade_rc*)
- *interpretación de perceptos* (*lógica y bayesiana*)
- *bloques de intensidad para probabilidades* (*bloque_prob*)

Esta separación evita duplicación y mantiene consistencia visual en todas las partes del proyecto.

3.2. Integración final del proyecto

La ejecución global se organiza mediante un script principal que actúa como menú y punto de entrada para palacio lógico, palacio bayesiano y río MDP, manteniendo modularidad y una experiencia unificada.

```
=====
                        BUSCANDO AL CORONEL KURTZ
                Fundamentos de Inteligencia Artificial
=====

Misión del Capitán Willard:
  1. Infiltrarse en el palacio
  2. Localizar al Coronel Kurtz
  3. Escapar con vida

Selecciona la parte del proyecto:
[1] Parte 1: Agente Lógico
[2] Parte 2: Incertidumbre
[0] Salir
```

Captura del menú principal en kurtz.py.

4. Error y constantes

Las **constantes de estilo** (colores, formatos) se encapsulan en *EstilosConsola*, evitando hardcodear códigos ANSI y manteniendo consistencia visual.

La finalización segura se gestiona con banderas/retornos estructurados (estado final + mensaje), evitando errores de control de flujo y permitiendo reportar claramente el motivo de fin de partida.

5. Bibliografía

Enunciado del proyecto (Parte 1 y 2). Documento interno de la asignatura.

Presentaciones de la asignatura: Temas 3, 4 y 5.

NumPy Documentation (guía y referencia).

OpenAI. (herramientas para análisis y creación de tablas/gráficas).